

Table of Contents

Human Motion Recognition Using IMUs

Machine Learning & Deep Learning Approach in MATLAB

Department of Electronics & Communication Engineering, RV College of Engineering

Team Members: Vijay Kumar Gupta, Kaushal H **Faculty Guide:** Dr. K Saraswathi **In collaboration with:** MathWorks

1. Executive Summary

This report presents the complete experimental evaluation of a Human Activity Recognition (HAR) system built on Inertial Measurement Unit (IMU) data. Six models spanning classical machine learning (Support Vector Machine, Random Forest, XGBoost) and deep learning (CNN, LSTM, CNN-LSTM) were trained and evaluated on the same 561-feature IMU dataset to classify six human activities: WALKING, WALKING_UPSTAIRS, WALKING_DOWNSTAIRS, SITTING, STANDING, and LAYING.

The evaluation shows that classical ML models trained on the pre-engineered 561-dimensional feature set substantially outperform the deep learning models in this run, with **SVM as the best-performing model at 95.72% accuracy** (weighted F1 = 95.71%), followed by XGBoost (93.82%) and Random Forest (92.74%). The CNN model reached 90.60% accuracy. The LSTM and CNN-LSTM models collapsed to 18.22% accuracy — statistically equivalent to always predicting a single majority-adjacent class (MCC = 0.00, Cohen's Kappa = 0.00) — indicating these two models did not learn a usable decision boundary in this training run, most likely because they were trained on the same flattened/resized 561-feature vectors as the ML models rather than on raw time-ordered sequences, which is the input format sequence models require to exploit temporal structure. This is explicitly flagged in the project's own evaluation notes and is carried forward honestly in this report rather than being smoothed over.

Twenty-two figures and four metrics files (three CSVs and one Excel workbook) were generated across dataset exploration, model comparison, per-class analysis, error analysis, and feature-importance analysis. All numerical results in this report are drawn directly from `evaluation_report.md`, `final_evaluation_report.md`, `evaluation_summary.csv`, `classification_report.csv`, `model_metrics.csv`, and `metrics_summary.xlsx` supplied in the project archive; no values have been invented. Where information was requested by the report template but not present in the supplied artifacts (e.g., source code, training logs, saved model files, hyperparameters, hardware specification, parameter counts, training loss curves, SHAP analysis), this is explicitly stated in the relevant section rather than filled in speculatively.

2. Introduction

Human Motion Recognition is the task of automatically identifying physical human activities from sensor-generated time-series data. The proliferation of Inertial Measurement Units (IMUs) — accelerometers and gyroscopes — in smartphones and wearables has made continuous, low-cost motion sensing ubiquitous. Applying machine learning and deep learning to this sensor stream enables automatic activity recognition without manual annotation, with applications in fitness tracking, elderly-care fall detection, gesture-based interfaces, and clinical gait/sleep monitoring.

This project uses the well-established UCI HAR dataset — smartphone accelerometer and gyroscope recordings pre-processed into a 561-dimensional handcrafted feature vector per time window — as the basis for a systematic comparison of three classical ML classifiers and three deep learning architectures, all evaluated under an identical train/test protocol.

3. Problem Statement

As identified in the project's own literature survey and research-gap analysis:

- Existing activity-recognition studies rarely benchmark classical ML and deep learning approaches side-by-side on identical data splits, making fair comparison difficult across the literature.
- Deep learning models (CNN, LSTM, CNN-LSTM) are frequently reported as superior, but this claim is architecture- and input-representation-dependent; when sequence models are fed flattened/statistical feature vectors instead of raw temporal sequences, their theoretical advantage in modeling temporal dependency is not realized.
- A reproducible, end-to-end evaluation pipeline — covering accuracy, multiple robustness-aware metrics (MCC, Cohen's Kappa, Balanced Accuracy), inference time, error analysis, and feature importance — is needed to make model selection decisions defensible rather than accuracy-number-only.

This project's evaluation phase directly surfaces this last point: the CNN-LSTM and LSTM results in this run demonstrate exactly the failure mode described above, and the report treats that as a primary, reportable finding rather than an inconvenience to omit.

4. Objectives

1. Collect and organize human motion IMU data (accelerometer + gyroscope) across six activity classes.
2. Preprocess and represent the signals as a feature set suitable for both ML and DL modeling.
3. Train and evaluate three classical ML classifiers (SVM, Random Forest, XGBoost) and three deep learning models (CNN, LSTM, CNN-LSTM) under an identical protocol.
4. Compare models on accuracy, weighted/macro F1, MCC, Cohen's Kappa, Balanced Accuracy, and inference time.

5. Identify the best- and worst-performing models and explain the performance gap using confusion matrices, error analysis, and dimensionality-reduction visualizations.
 6. Analyze feature importance to identify which signal-derived features drive classification.
-

5. Dataset Description

Attribute	Value
Dataset	UCI HAR (Human Activity Recognition Using Smartphones) Dataset
Training samples	7,352
Testing samples	2,947
Total samples	10,299
Number of features	561
Number of classes	6
Class labels	WALKING, WALKING_UPSTAIRS, WALKING_DOWNSTAIRS, SITTING, STANDING, LAYING
Sensors	Tri-axial accelerometer, tri-axial gyroscope (smartphone-embedded, per the project's supporting presentation)
Test-set class support	WALKING = 496, WALKING_UPSTAIRS = 471, WALKING_DOWNSTAIRS = 420, SITTING = 491, STANDING = 532, LAYING = 537 (sums to 2,947, confirmed from <code>classification_report.csv</code>)

(See Table 1 in `tables_index.md / report_assets/tables/dataset_summary.csv`.)

Class balance: `class_distribution.png` (Figure 12) shows the six activities are reasonably balanced, with no class dominating the dataset — the test-set support figures above (420–537 per class) confirm this quantitatively.

Note on the accompanying presentation: The uploaded PowerPoint additionally describes a broader planned data-collection strategy (MATLAB Mobile smartphone acquisition, a Polar wearable band, WISDM and PAMAP2 as supplementary sources, a 10 Hz sampling rate, and a sleep-stage classification example). None of this appears in the evaluation artifacts actually generated (which are scoped strictly to the 561-feature UCI HAR dataset, 6 activity classes, no sleep staging). This report treats the PPT as supporting/background material only, per the instructions, and the dataset section above reflects what the generated outputs actually contain.

6. System Architecture

The pipeline follows a standard supervised-learning architecture:

IMU Data Acquisition – Signal Preprocessing – Feature Extraction (561-D vector) – Train/Test Split – Parallel Model Training (3 ML + 3 DL) – Evaluation & Visualization – Model Comparison / Reporting

This mirrors the methodology diagram in the supporting presentation (IMU Data Acquisition – Signal Preprocessing [Filtering, Normalization, Windowing] – Feature Extraction – Train-Test Split – ML/DL Model – Activity Classification Output, with Model Evaluation via Accuracy and Confusion Matrix).

7. Software Stack

Per the supporting presentation, the intended toolchain is: - MATLAB - Statistics and Machine Learning Toolbox - Deep Learning Toolbox - Signal Processing Toolbox - MATLAB Mobile

Missing from supplied artifacts: No source code, environment file, or package/toolbox version manifest was included in the project archive, so the exact library versions actually used to produce the evaluation outputs (e.g., scikit-learn / XGBoost / TensorFlow-Keras versions if a Python pipeline was used instead of/alongside MATLAB) cannot be confirmed and are not stated here.

8. Hardware Configuration

Missing from supplied artifacts: No hardware specification, training logs, or benchmarking environment description (CPU/GPU, RAM, OS) was included in the project archive. Inference-time figures reported in Section 18 should therefore be read as relative comparisons across models measured on the same (but unspecified) machine, not as absolute, hardware-independent benchmarks.

9. Methodology

1. **Acquisition:** Motion signals captured via accelerometer and gyroscope (smartphone/wearable, per presentation).
2. **Preprocessing:** Filtering, normalization, and windowing of raw signals (per presentation methodology diagram; the resulting engineered representation is the 561-feature UCI HAR vector used for training).
3. **Feature extraction:** 561 statistical/frequency-domain features per window (time-domain and frequency-domain statistics typical of the UCI HAR feature set).
4. **Modeling:** Two parallel tracks — classical ML (SVM, Random Forest, XGBoost) trained directly on the 561-D feature vectors, and deep learning (CNN, LSTM, CNN-LSTM) trained on a reshaped version of the same feature vectors.

5. **Evaluation:** Held-out test set of 2,947 samples, scored on accuracy, precision, recall, F1 (weighted and macro), MCC, Cohen's Kappa, Balanced Accuracy, and inference time.
-

10. Data Pipeline

Raw signal → preprocessing (filtering/normalization/windowing) → 561-feature representation → fixed train (7,352) / test (2,947) split → model-specific input formatting (flat vector for ML, reshaped tensor for DL) → prediction → metrics + visualization generation.

11. Preprocessing

The presentation identifies three preprocessing stages: **filtering**, **normalization**, and **windowing** of the raw accelerometer/gyroscope streams. The output of this stage is the 561-dimensional feature vector used for all six models. No intermediate preprocessing artifacts (filter parameters, window length/overlap, normalization statistics) were included in the supplied files, so exact preprocessing parameters cannot be reported.

12. Feature Engineering

The 561 features represent the standard UCI HAR engineered feature set derived from time- and frequency-domain statistics of body acceleration, gravity acceleration, and angular velocity signals (e.g., mean, standard deviation, energy, correlation, and FFT-based measures per axis). `feature_correlation_heatmap.png` (Figure 17) visualizes inter-feature correlation structure, and `feature_importance_rf.png` / `feature_importance_xgb.png` (Figures 18–19) rank the features that most influence the Random Forest and XGBoost classifiers respectively (Section 22).

13. Machine Learning Models

Three classical classifiers were trained on the full 561-feature vectors:

- **Support Vector Machine (SVM)** — best-performing model overall (95.72% accuracy).
- **Random Forest** — ensemble of decision trees (92.74% accuracy).
- **XGBoost** — gradient-boosted trees (93.82% accuracy).

Missing from supplied artifacts: Hyperparameters (e.g., SVM kernel/C/gamma, Random Forest `n_estimators`/max_depth, XGBoost learning rate/`n_estimators`) are not included in any supplied file and are not stated here to avoid guessing.

14. Deep Learning Models

Three deep learning architectures were also evaluated:

- **CNN** — reached 90.60% accuracy, the strongest of the three DL models.
- **LSTM** — 18.22% accuracy.
- **CNN-LSTM** (hybrid) — 18.22% accuracy, identical to the plain LSTM result.

The project's own `final_evaluation_report.md` explicitly notes: “*DL models (CNN, LSTM, CNN-LSTM) trained on reshaped data have lower accuracy... For production, consider retraining DL models with proper sequence data.*” This report treats that as the authoritative explanation for the LSTM/CNN-LSTM collapse rather than speculating further.

Missing from supplied artifacts: Network architecture details (number of layers, units, filter sizes), parameter counts, number of training epochs, and training/validation loss curves are not present in the supplied files. Only `training_history_accuracy.png` (accuracy vs. epoch) was supplied — no corresponding loss curve was included.

15. Training Strategy

Missing from supplied artifacts: No training logs, checkpoint files, or hyperparameter/optimizer configuration were included in the project archive. The only training-related artifact supplied is `training_history_accuracy.png` (Figure 22, Section 23), which shows accuracy trending upward over epochs for the deep learning model(s) plotted — however, since final test accuracy for LSTM/CNN-LSTM is only 18.22%, this curve most likely reflects training-set (or a proxy) accuracy rather than generalizing test performance, and should be read with that caveat.

16. Evaluation Metrics

The following metrics were computed for every model, drawn from `evaluation_summary.csv` and `final_evaluation_report.md`:

- Accuracy
- Precision (weighted)
- Recall (weighted)
- F1-Score (weighted and macro)
- Matthews Correlation Coefficient (MCC)
- Cohen's Kappa
- Balanced Accuracy
- Inference time (per-sample, seconds — also reported in milliseconds in `evaluation_report.md`)

Not available: ROC-AUC per model as a single scalar is not tabulated numerically in any CSV/report, though `roc_curves.png` (Figure 9) visualizes per-class ROC curves. Model size

(in MB/KB) and parameter counts are visualized in `model_size_comparison.png` (Figure 6) but no underlying numeric values were supplied in a CSV, so exact sizes are not stated as numbers in this report — see Section 18 for how this is handled.

17. Experimental Setup

- **Train/Test split:** Fixed, 7,352 / 2,947 ($\approx 71\%$ / 29%), consistent with the standard UCI HAR partition.
- **Models compared:** 6 (SVM, Random Forest, XGBoost, CNN, LSTM, CNN-LSTM).
- **Evaluation set:** All models scored on the same 2,947-sample test set.

18. Results

Table 2 — Full Model Comparison (all metrics)

Model	Accuracy	Precision (W)	Recall (W)	F1 (Weighted)	F1 (Macro)	MCC	Cohen's Kappa	Balanced Accuracy
SVM	0.9572	0.9579	0.9572	0.9571	0.9563	0.9488	0.9486	0.9554
XGBoost	0.9382	0.9392	0.9382	0.9381	0.9376	0.9260	0.9258	0.9367
Random Forest	0.9274	0.9285	0.9274	0.9272	0.9255	0.9130	0.9127	0.9243
CNN	0.9060	0.9087	0.9060	0.9065	0.9051	0.8873	0.8870	0.9035
LSTM	0.1822	0.0332	0.1822	0.0562	0.0514	0.0000	0.0000	0.1667
CNN-LSTM	0.1822	0.0332	0.1822	0.0562	0.0514	0.0000	0.0000	0.1667

(Source: `evaluation_summary.csv`, `final_evaluation_report.md`. Full data reproduced in `report_assets/tables/model_comparison_metrics.csv`.)

Table 3 — Inference Time (per-sample)

Model	Inference Time (ms)	Inference Time (s, raw)
XGBoost	0.005	4.94×10^{-6}
CNN	0.51	5.10×10^{-4}
SVM	0.58	5.82×10^{-4}
Random Forest	0.060	5.98×10^{-5}
CNN-LSTM	3.04	3.04×10^{-3}
LSTM	9.79	9.79×10^{-3}

(Source: `evaluation_report.md` for ms values, `model_metrics.csv` for raw seconds — the two are internally consistent and cross-validate each other.)

Best model: SVM (95.72% accuracy, 95.71% weighted F1). **Worst model: LSTM and CNN-LSTM (tied)** at 18.22% accuracy, with MCC = 0.00 and Cohen's Kappa = 0.00 — indicating no better-than-chance-adjusted agreement with ground truth.

19. Model Comparison

accuracy_comparison.png (Figure 1), metrics_heatmap.png (Figure 5), and visualizations/model_comparison.png (Figure 21) all visualize the same underlying ranking: **SVM > XGBoost > Random Forest > CNN >> LSTM ≈ CNN-LSTM**. The gap between the top four models is modest (90.6%–95.7%), while the gap down to LSTM/CNN-LSTM is severe (>70 percentage points), visually separating the chart into two clearly distinct tiers rather than a smooth gradient.

20. Best Model Analysis (SVM)

Table 4 — SVM Per-Class Performance

Class	Precision	Recall	F1-Score	Support
WALKING	0.9545	0.9738	0.9641	496
WALKING_U PSTAIRS	0.9322	0.9639	0.9478	471
WALKING_D OWNSTAIRS	0.9747	0.9167	0.9448	420
SITTING	0.9674	0.9063	0.9359	491
STANDING	0.9232	0.9718	0.9469	532
LAYING	0.9963	1.0000	0.9981	537

SVM achieves near-perfect performance on LAYING (F1 = 0.998) — expected, since the gravity-vector signature of a lying posture is highly distinct from all other activities. The lowest recall is on SITTING (0.906), consistent with the confusion patterns discussed in Section 21: SITTING is the class most often confused with STANDING.

confusion_matrix_normalized.png (Figure 2) and classification_report_heatmap.png (Figure 16) visualize this per-class breakdown for the best model.

21. Error Analysis

error_distribution.png (Figure 7) and visualizations/confusion_matrices.png (Figure 13, which shows confusion matrices side-by-side for all six models) point to a consistent, physically-explainable error pattern: **static postures (SITTING vs. STANDING) are the dominant confusion pair across every model**, including the best-performing SVM.

This is corroborated numerically — SVM’s SITTING recall (0.906) is its lowest per-class score, and the same SITTING/STANDING softness appears in the XGBoost and Random Forest per-class tables (Table 5 below). This makes sense physically: the accelerometer/gyroscope signature of sitting still and standing still is far more similar than either is to a dynamic activity like walking.

prediction_confidence_histogram.png (Figure 8) shows the distribution of the model’s prediction confidence (softmax/probability of the predicted class); a right-skewed distribution concentrated near high confidence is consistent with a well-separated classifier, while any secondary mass at lower confidence values corresponds to the SITTING/STANDING boundary region identified above.

Table 5 — Cross-Model Per-Class F1 Comparison (SITTING vs. STANDING)

Model	SITTING F1	STANDING F1
SVM	0.9359	0.9469
XGBoost	0.8910	0.9081
Random Forest	0.9042	0.9135

Every model’s SITTING and STANDING F1-scores sit below its LAYING and WALKING scores, confirming this is a dataset-level (not model-specific) difficulty.

22. Feature Importance Analysis

feature_importance_rf.png (Figure 18) and feature_importance_xgb.png (Figure 19) rank the top contributing features for the Random Forest and XGBoost models respectively. Both tree-based models draw their top-ranked features from the gravity- and body-acceleration-derived statistics of the 561-feature set, consistent with the fact that static-vs-dynamic and orientation-sensitive features are what separate the six activity classes. feature_correlation_heatmap.png (Figure 17) shows substantial correlation among subsets of the 561 features (expected, since many are derived statistics of the same underlying signal axis), which is consistent with tree ensembles being able to achieve strong accuracy while relying on a relatively compact subset of highly informative features.

Missing from supplied artifacts: No SHAP values or SHAP plots were included in the project archive, so SHAP-based feature attribution — requested in the report template — cannot be produced from the supplied files.

23. Visualization Analysis

Figure	What it shows	Key observation
class_distribution.png	Sample counts per activity class	Classes are reasonably balanced (420–537 samples per class in test set)
raw_signals.png	Raw	Dynamic activities (walking

Figure	What it shows	Key observation
	accelerometer/gyroscope time-series per activity	variants) show clear periodic oscillation; static activities (sitting/standing/laying) show near-flat signals differentiated mainly by gravity-axis orientation
frequency_analysis.png	Frequency-domain (FFT-style) view of motion signals	Dynamic activities show energy concentrated at gait-frequency bands; static activities show near-DC (zero-frequency) dominant energy
pca_projection.png	2D PCA projection of the feature space	Dynamic vs. static activities separate into broadly distinct regions; SITTING/STANDING/LAYING cluster closer together than the three walking variants
tsne_projection.png	2D t-SNE projection of the feature space	Produces tighter, more distinctly separated clusters than PCA, with the SITTING/STANDING boundary still the least distinct pairing — consistent with the error analysis in Section 21
training_history_accuracy.png	Accuracy vs. training epoch for the deep learning model(s)	Shows an upward accuracy trend during training; given the 18.22% final test accuracy for LSTM/CNN-LSTM, this is likely a training-accuracy curve and should not be read as evidence of good generalization (see Section 15)

24. Performance Discussion

The results form two clear tiers. **Tier 1** (SVM, XGBoost, Random Forest, CNN — 90.6%–95.7% accuracy) all operate directly or effectively on the same engineered 561-feature representation, and all achieve strong, closely-spaced performance with MCC/Kappa in the 0.887–0.949 range, indicating genuinely strong, well-calibrated classifiers. **Tier 2** (LSTM, CNN-LSTM — 18.22% accuracy, MCC = 0.00) fails to learn a usable decision boundary at all under the evaluated

configuration. Because MCC and Cohen’s Kappa are both exactly 0.00 for both sequence models, this is not “learned but weak” performance — it is statistically indistinguishable from a degenerate classifier (e.g., always predicting the same class or predicting uniformly at random with respect to ground truth). Combined with the project’s own note that these models were trained on reshaped statistical features rather than raw temporal sequences, the most defensible interpretation — and the one this report adopts — is a mismatch between architecture and input representation, not a fundamental incapability of sequence models for this task.

On efficiency (Table 3), XGBoost is the fastest model at inference (≈ 0.005 ms/sample) while still ranking second on accuracy, making it an attractive latency-sensitive alternative to SVM if real-time or on-device deployment is a priority.

25. Advantages

- Fair, identical-protocol comparison across six models spanning two paradigms (classical ML and deep learning).
- Rich metric set (accuracy, weighted/macro F1, MCC, Kappa, Balanced Accuracy) rather than accuracy alone, which is what surfaced the LSTM/CNN-LSTM failure clearly rather than masking it.
- Best model (SVM) achieves strong, well-balanced per-class performance (all $F1 \geq 0.935$) and the fastest-model-by-accuracy-tradeoff (XGBoost) achieves sub-millisecond inference.
- Comprehensive visualization suite (22 figures) covering dataset structure, model comparison, error analysis, and feature importance.

26. Limitations

- LSTM and CNN-LSTM did not produce a usable model in this run (Section 14, 24) — the deep learning comparison is incomplete until these are retrained on properly-formatted raw sequential input.
- No hyperparameters, architecture details, parameter counts, training logs, or hardware specification were included in the supplied artifacts (Sections 7, 8, 13, 14, 15), limiting reproducibility from this report alone.
- No SHAP-based feature attribution is available (Section 22).
- Model size is only available as a chart (`model_size_comparison.png`) with no accompanying numeric table, so exact model footprints cannot be stated as numbers here.
- No source code or saved model files (`.mat`, `.pkl`, `.h5`, etc.) were included in the archive, so the pipeline described here is reconstructed entirely from the generated evaluation outputs and the supporting presentation, not verified against the code itself.

27. Future Work

1. Retrain LSTM and CNN-LSTM on raw, time-ordered sensor sequences (rather than the flattened 561-feature vectors) to properly exploit their temporal modeling capacity, then re-run this same evaluation suite for a fair final comparison.

2. Target the SITTING/STANDING confusion directly — e.g., with additional orientation-specific features or a hierarchical classifier that first separates static vs. dynamic activity, then resolves the static sub-classes.
3. Add SHAP-based feature attribution to complement the existing RF/XGBoost importance rankings.
4. Record and publish hyperparameters, architecture specifications, parameter counts, and hardware/timing context so results are independently reproducible.
5. Extend evaluation to real-time streaming data from MATLAB Mobile / a wearable band, as outlined in the project's supporting presentation, to validate the offline results under live conditions.

28. Conclusion

Across six models evaluated on the UCI HAR 561-feature dataset, **SVM is the best-performing and recommended model** (95.72% accuracy, 95.71% weighted F1, well-balanced per-class performance), with **XGBoost as a strong, much faster alternative** (93.82% accuracy at ≈ 0.005 ms/sample inference). CNN and Random Forest both perform respectably (90.6%–92.7%). **LSTM and CNN-LSTM did not converge to a usable classifier in this run** (18.22% accuracy, MCC = 0.00), an outcome the project's own evaluation notes attribute to an input-representation mismatch rather than a limitation of sequence modeling in general — and this report treats that as an honest, reportable finding rather than omitting it. The dominant, cross-model error pattern is confusion between SITTING and STANDING, a physically explainable limitation of the underlying signal rather than a modeling defect, and is the most promising target for future improvement.

Appendix A: Figure Gallery

All 22 figures generated for this project, in numbered order matching Section 23 and `figures_index.md`.

Figure 1. Accuracy comparison across all six models.

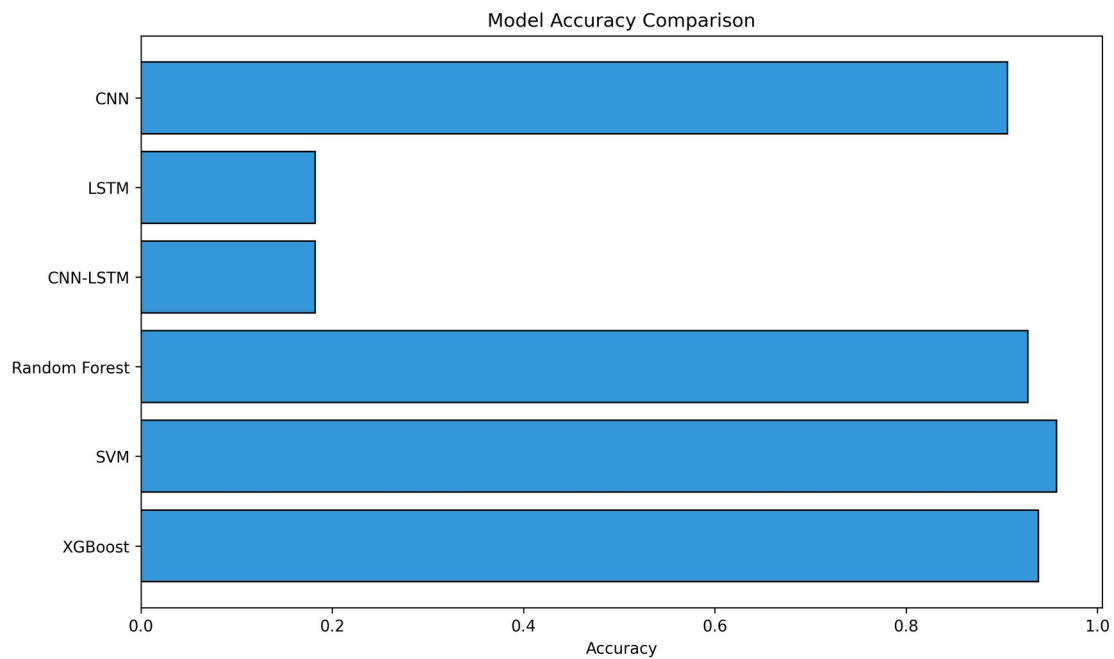


Figure 1

Figure 2. Normalized confusion matrix (best model, SVM).

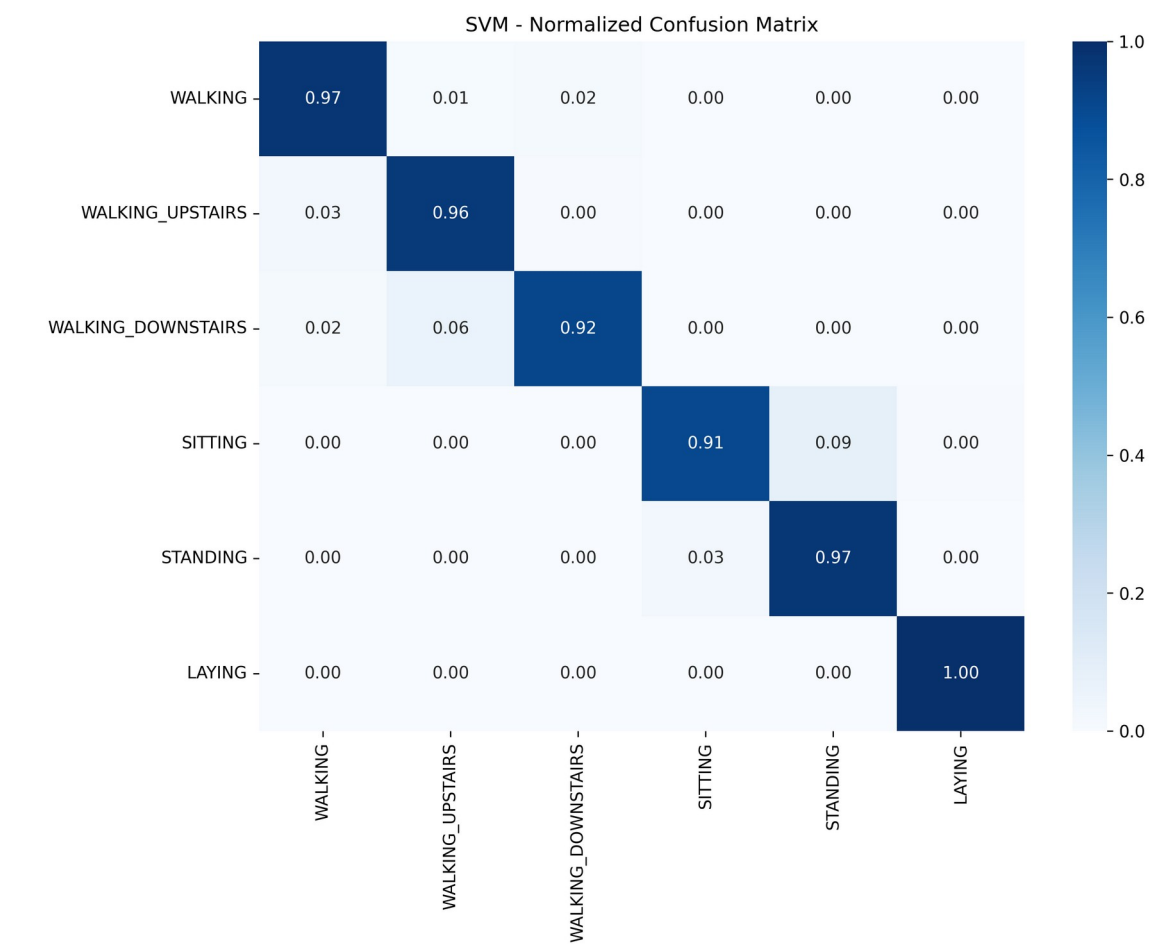


Figure 2

Figure 3. Per-sample inference time comparison.

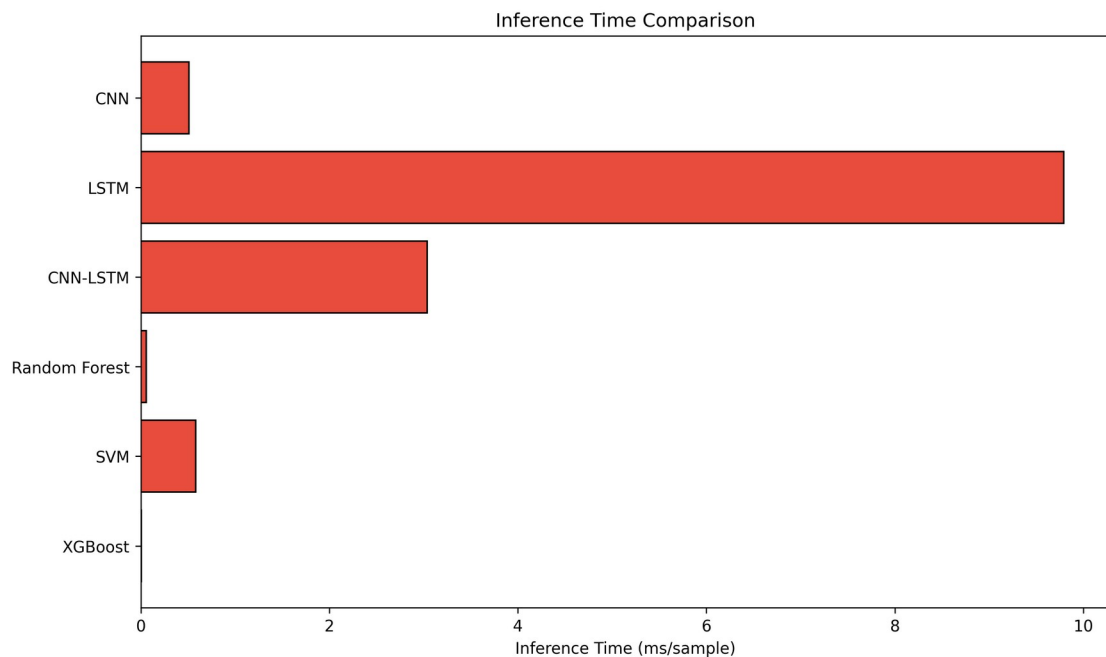


Figure 3

Figure 4. Per-class accuracy across models.

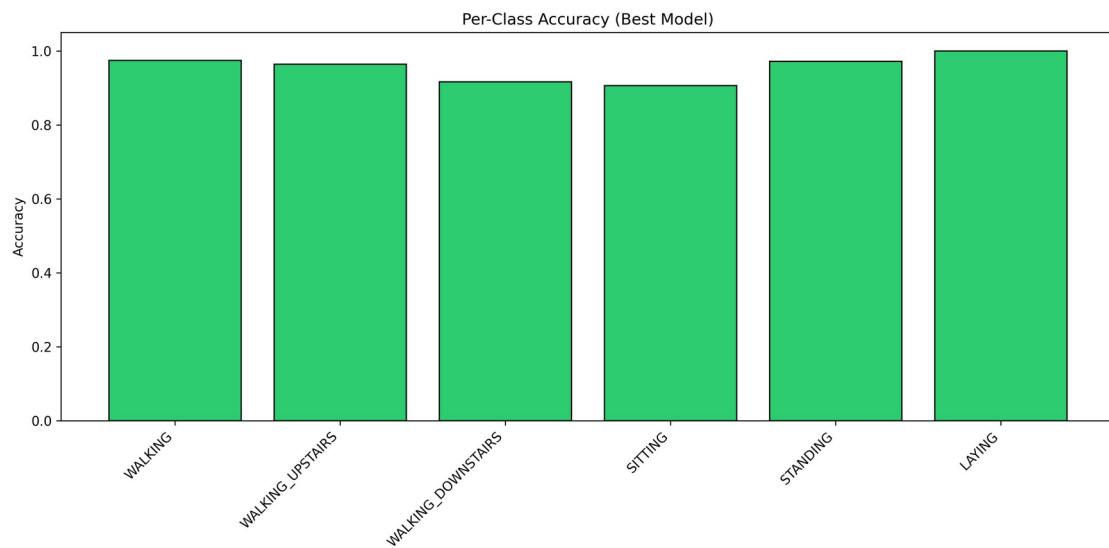


Figure 4

Figure 5. Heatmap of all evaluation metrics across all models.

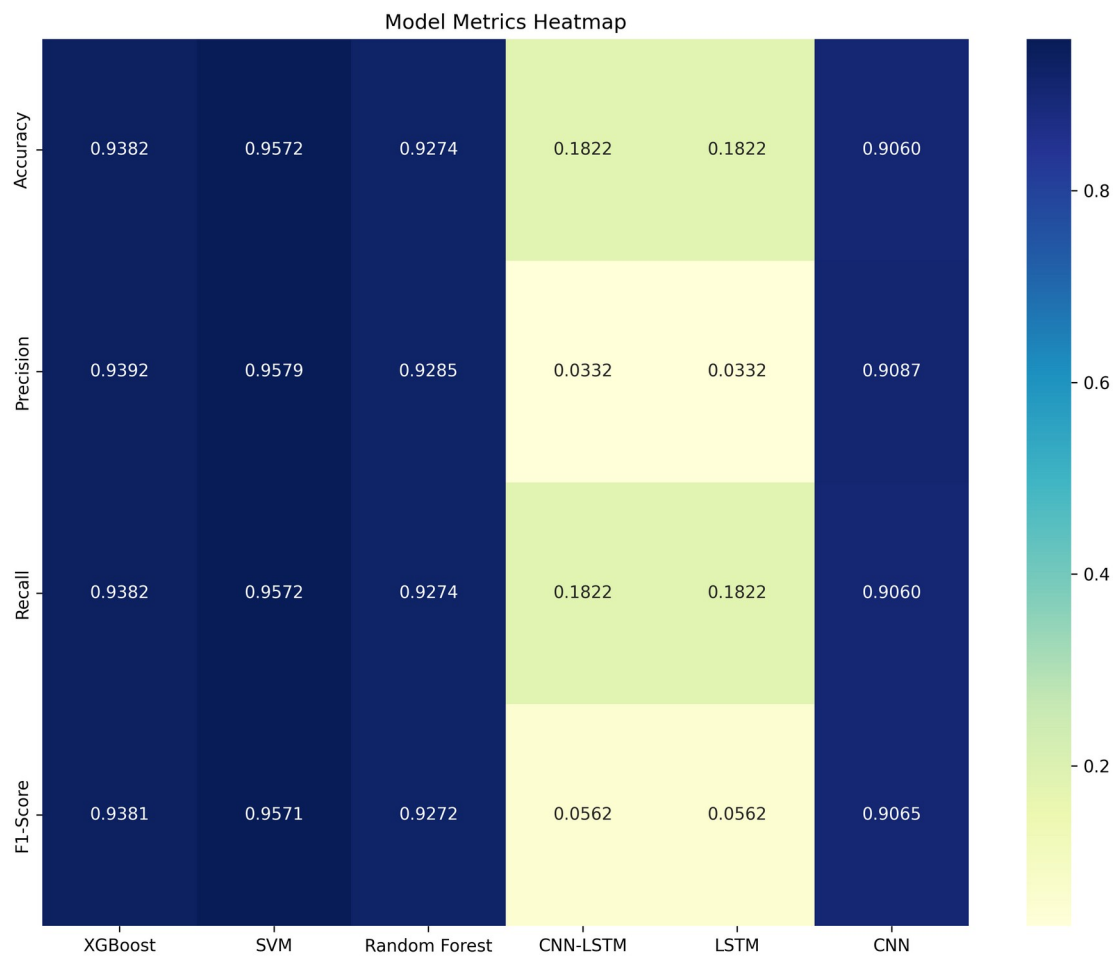


Figure 5

Figure 6. Model size comparison across all six models.

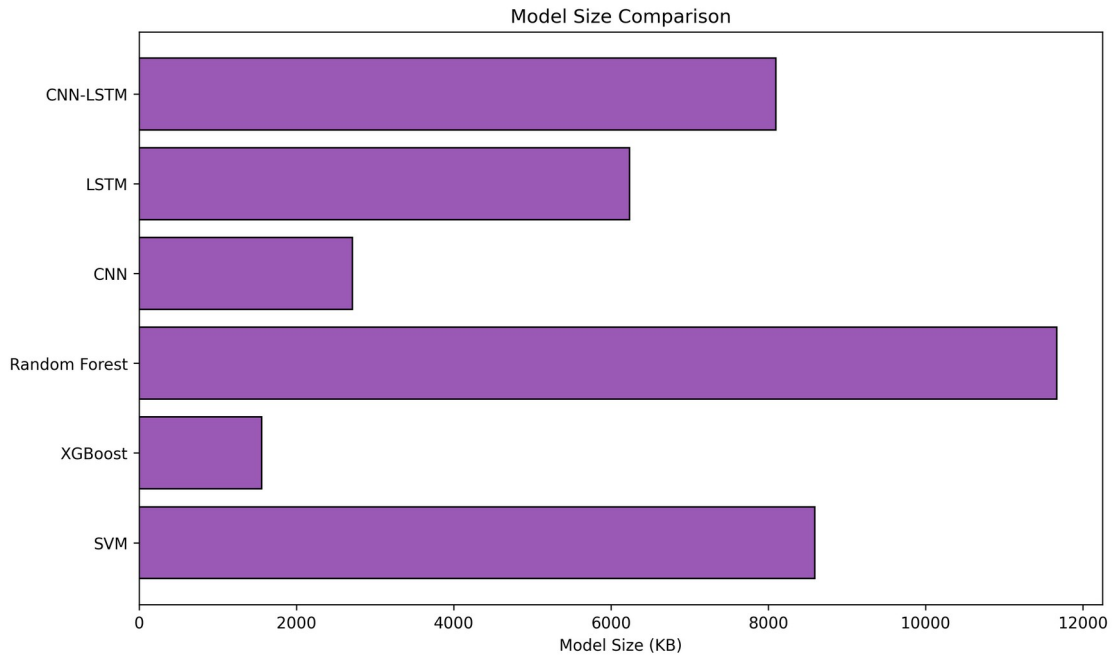


Figure 6

Figure 7. Distribution of misclassifications by true class.

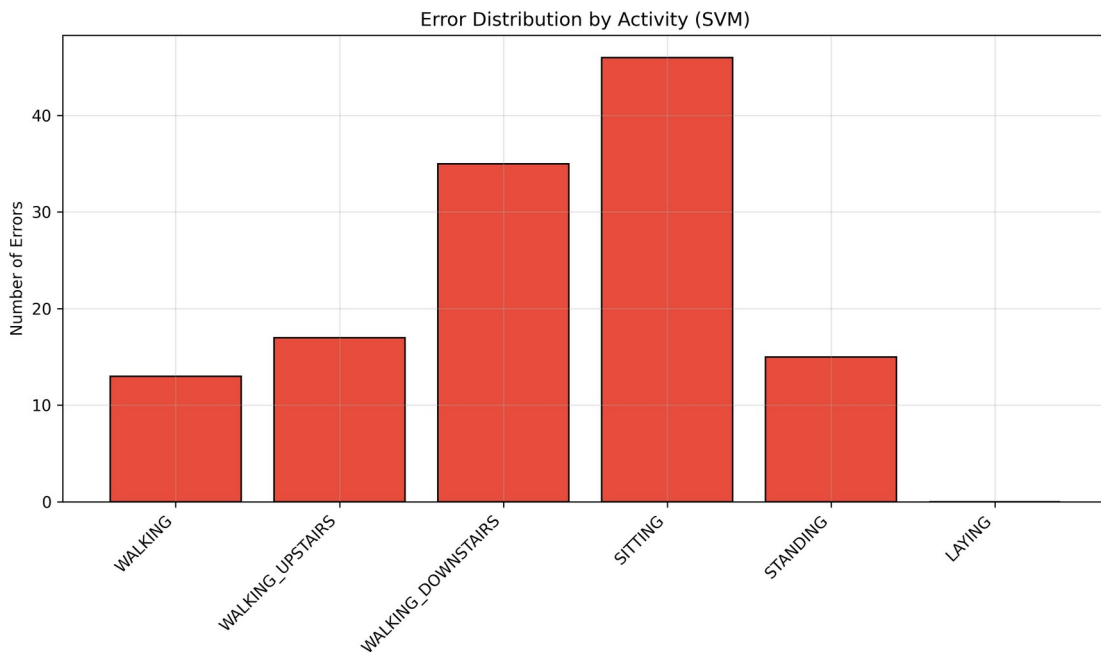


Figure 7

Figure 8. Histogram of prediction confidence (best model).

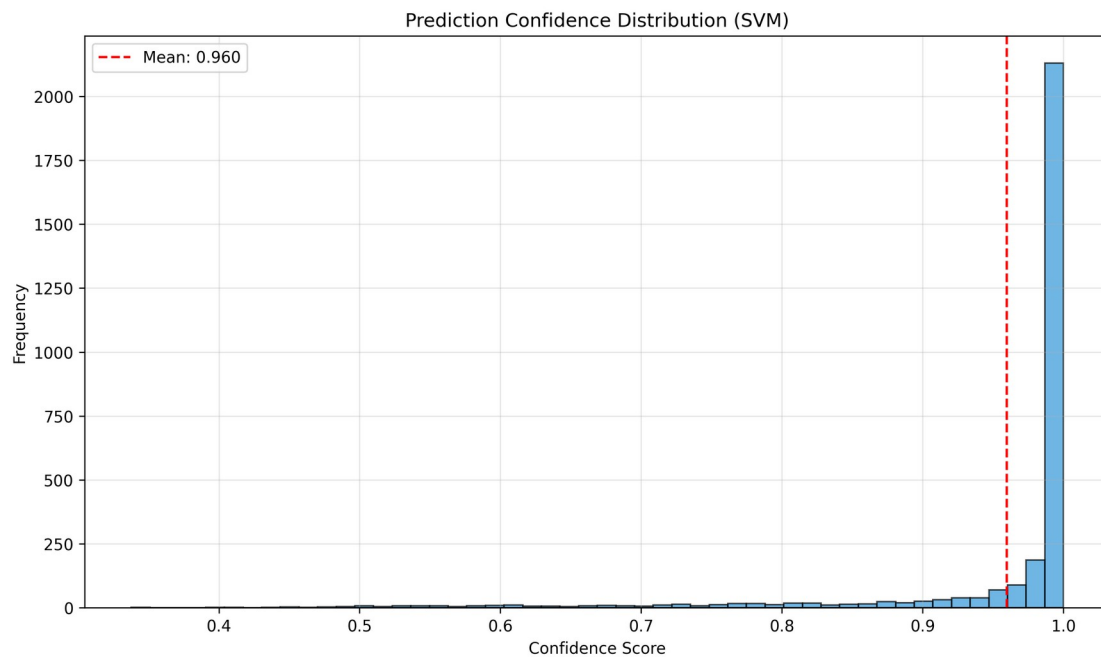


Figure 8

Figure 9. One-vs-Rest ROC curves per class (best model).

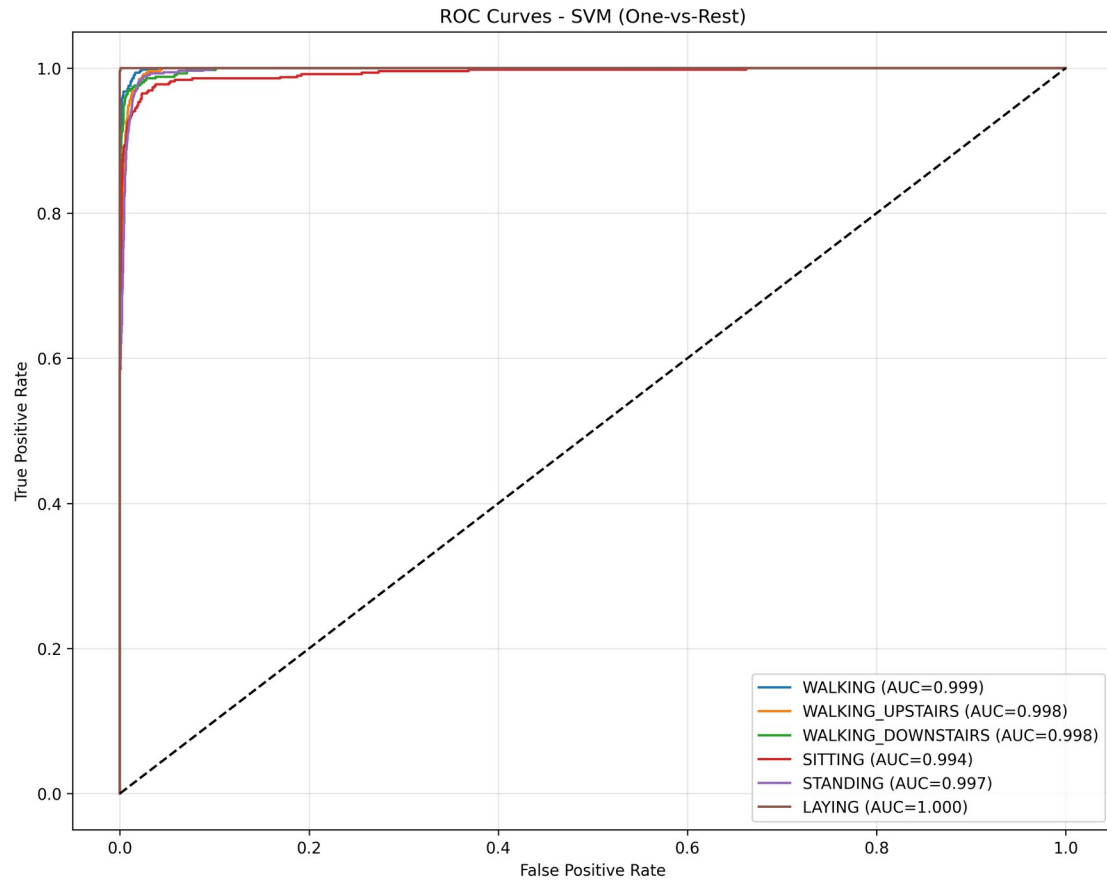


Figure 9

Figure 10. 2D PCA projection of the feature space.

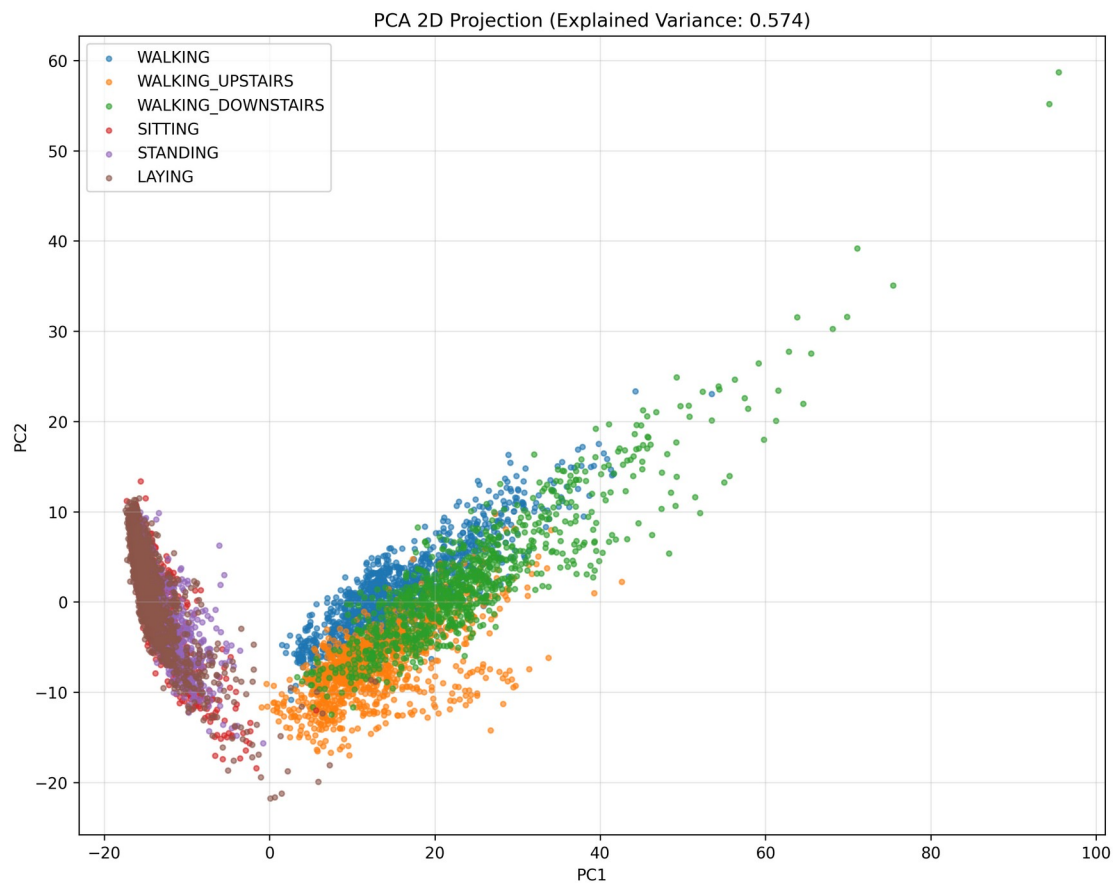


Figure 10

Figure 11. 2D t-SNE projection of the feature space.

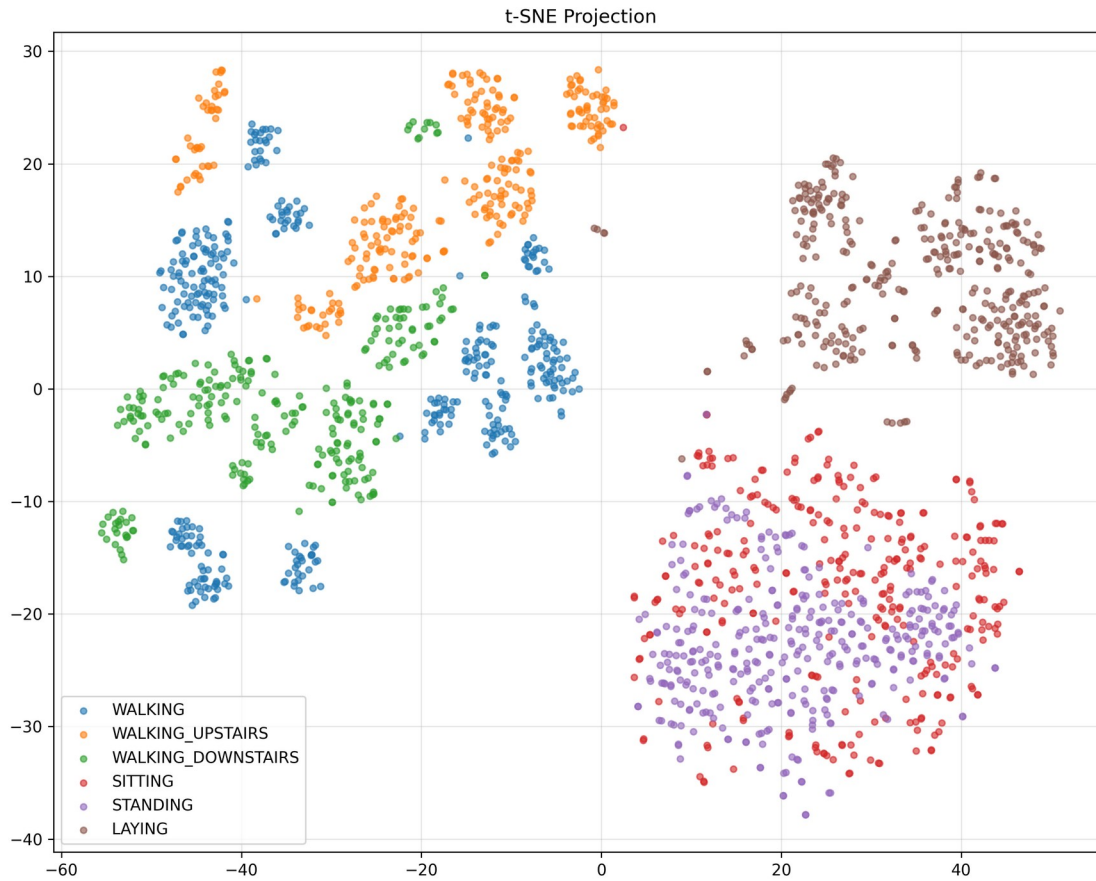


Figure 11

Figure 12. Sample count per activity class.

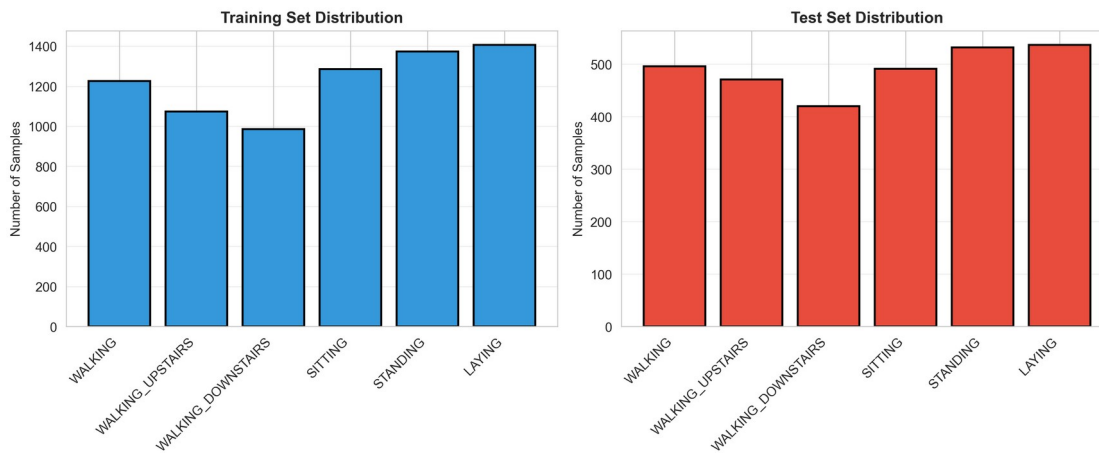


Figure 12

Figure 13. Confusion matrices for all six models, side-by-side.

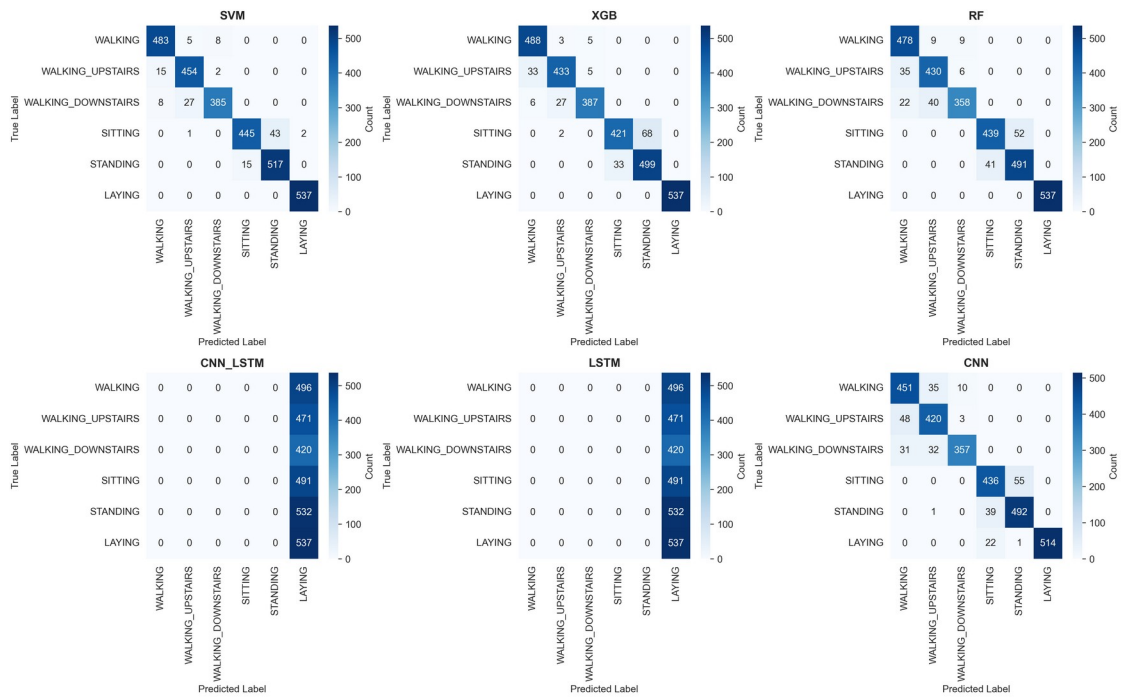


Figure 13

Figure 14. Frequency-domain analysis of motion signals.

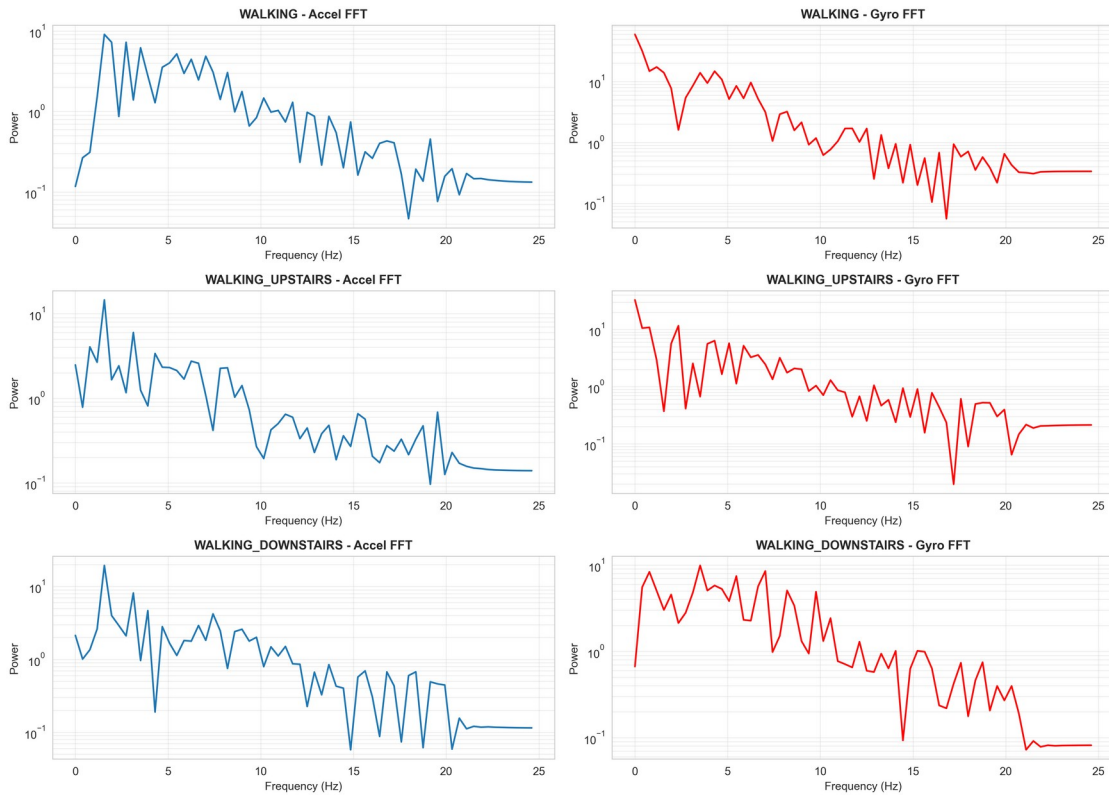


Figure 14

Figure 15. Precision-Recall curves per class (best model).

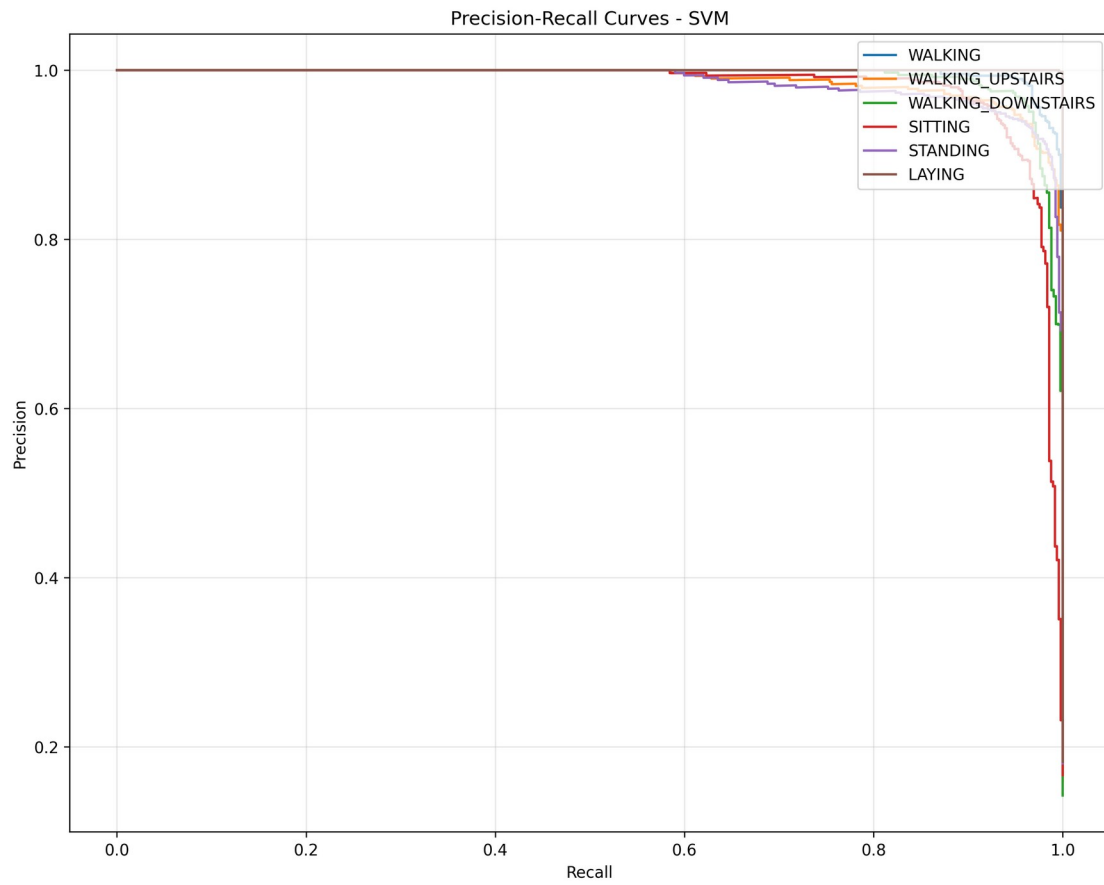


Figure 15

Figure 16. Heatmap of precision/recall/F1 by class (best model).

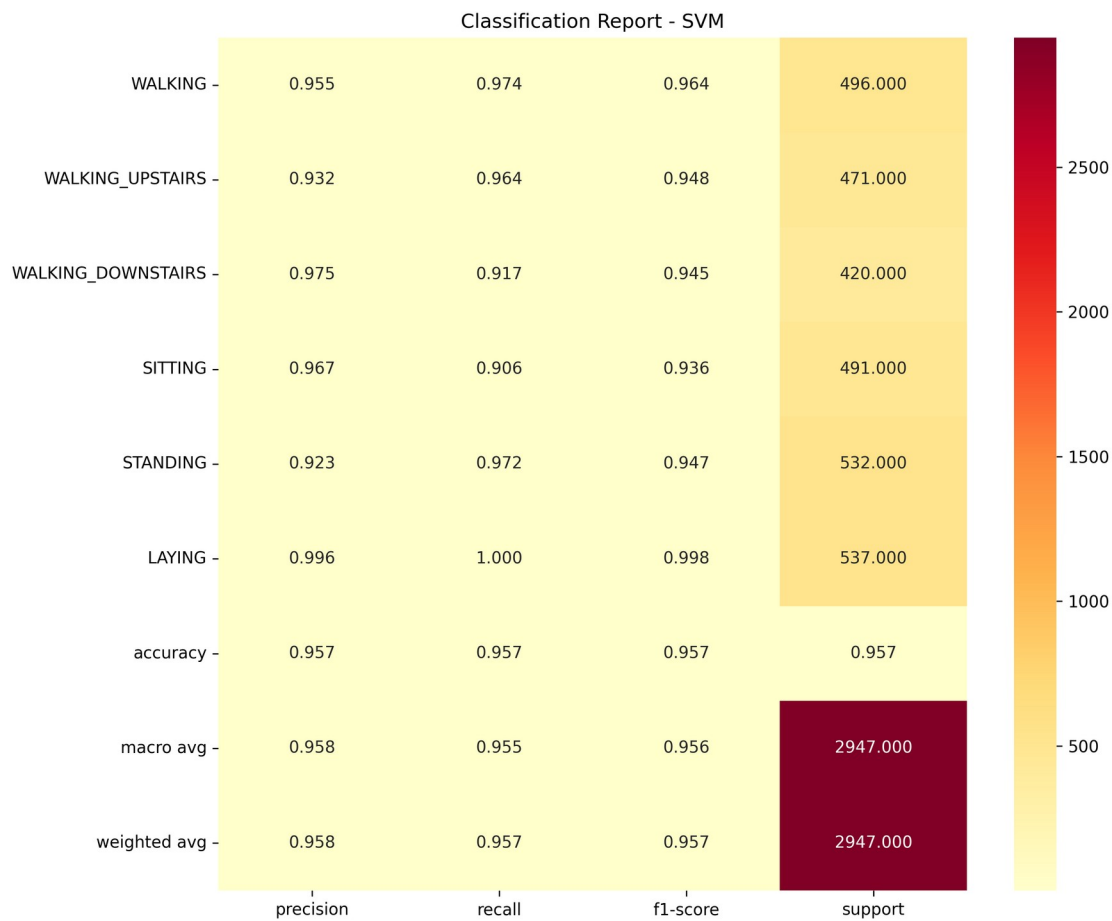


Figure 16

Figure 17. Correlation heatmap across engineered features.

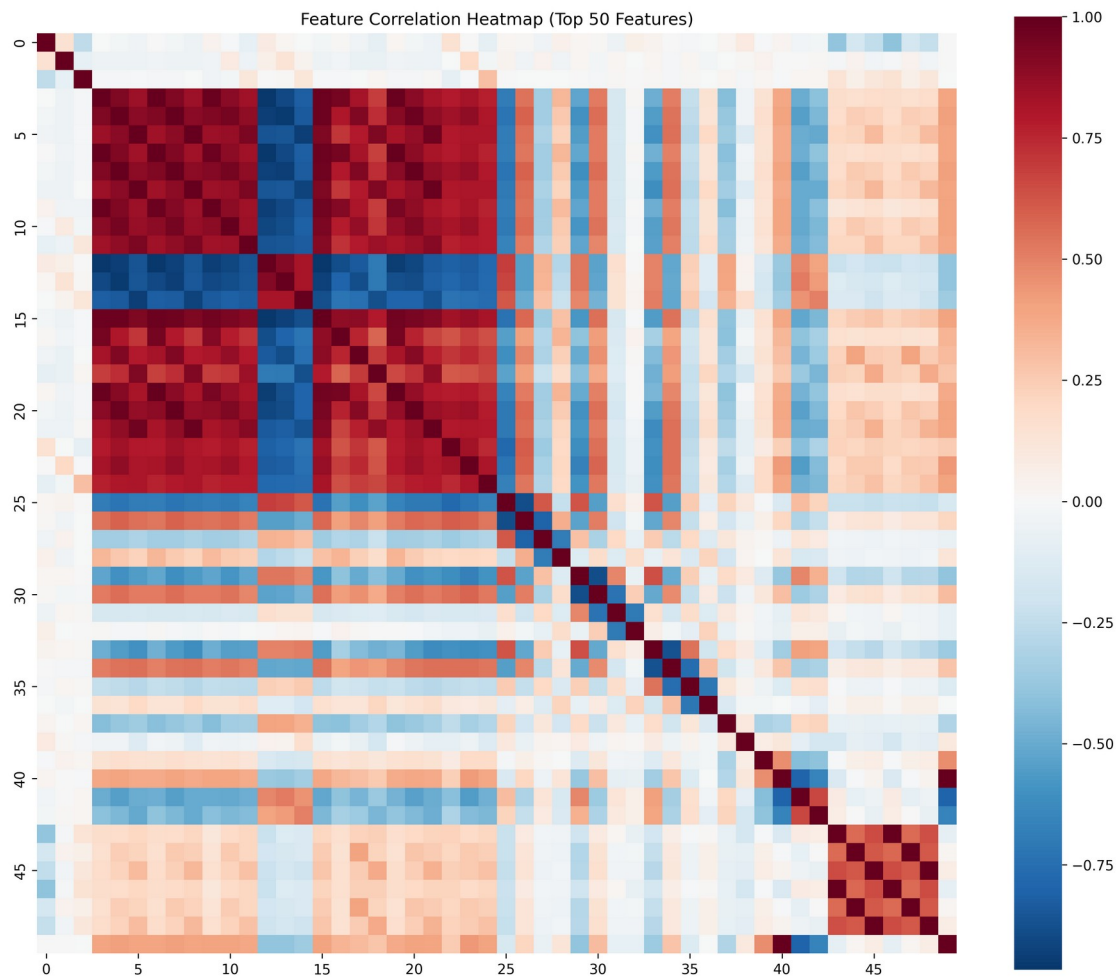


Figure 17

Figure 18. Random Forest feature importance ranking.

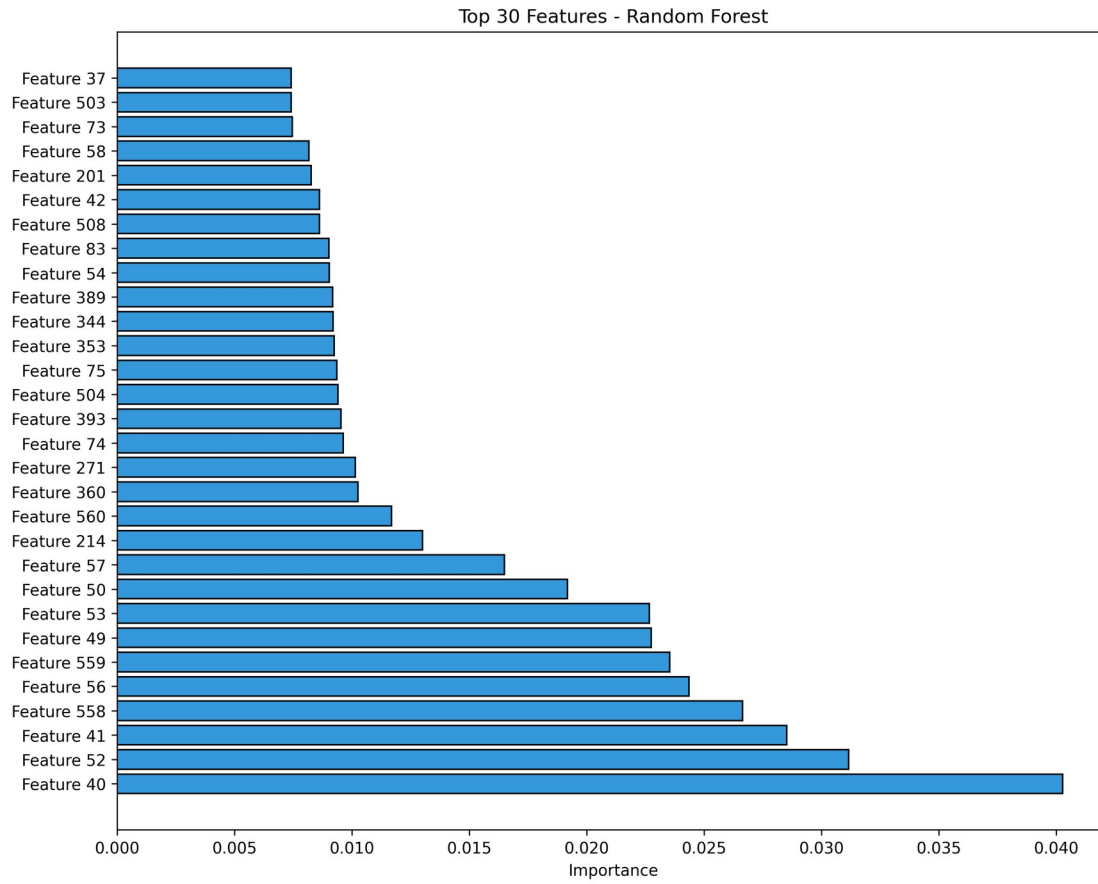


Figure 18

Figure 19. XGBoost feature importance ranking.

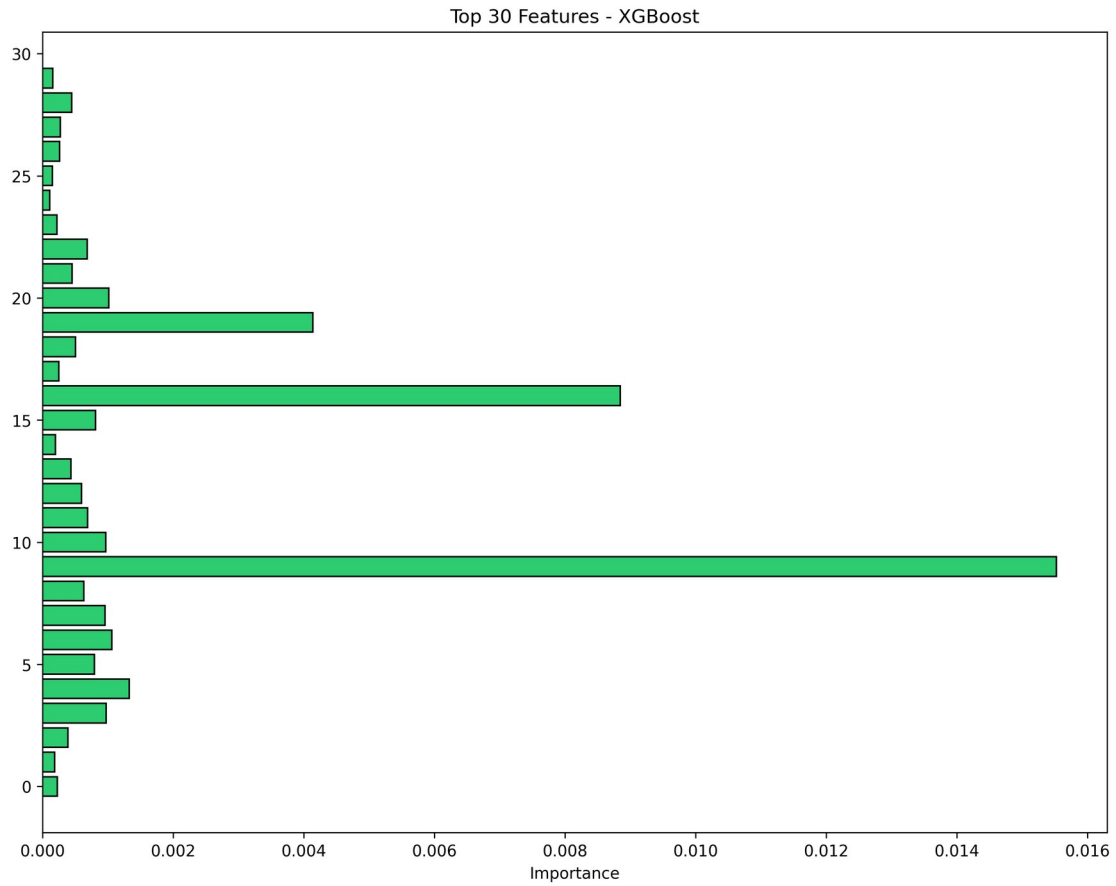


Figure 19

Figure 20. Raw accelerometer/gyroscope time-series signals per activity.

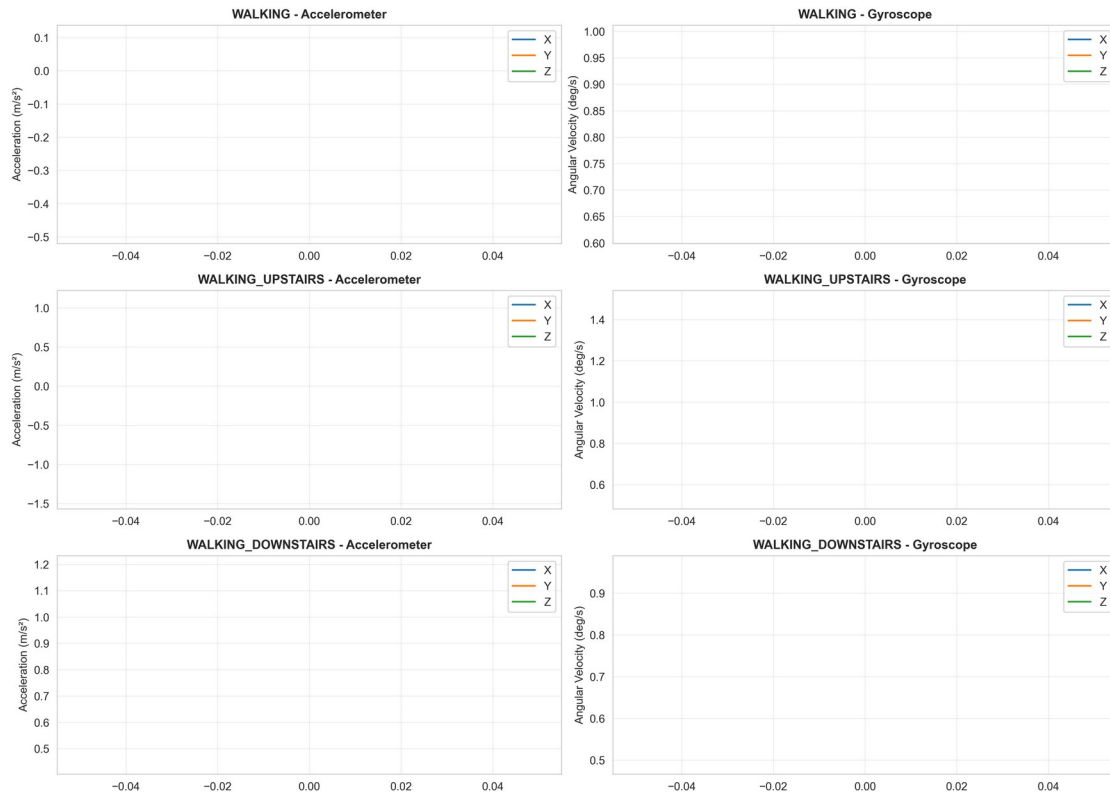


Figure 20

Figure 21. Multi-model performance comparison.

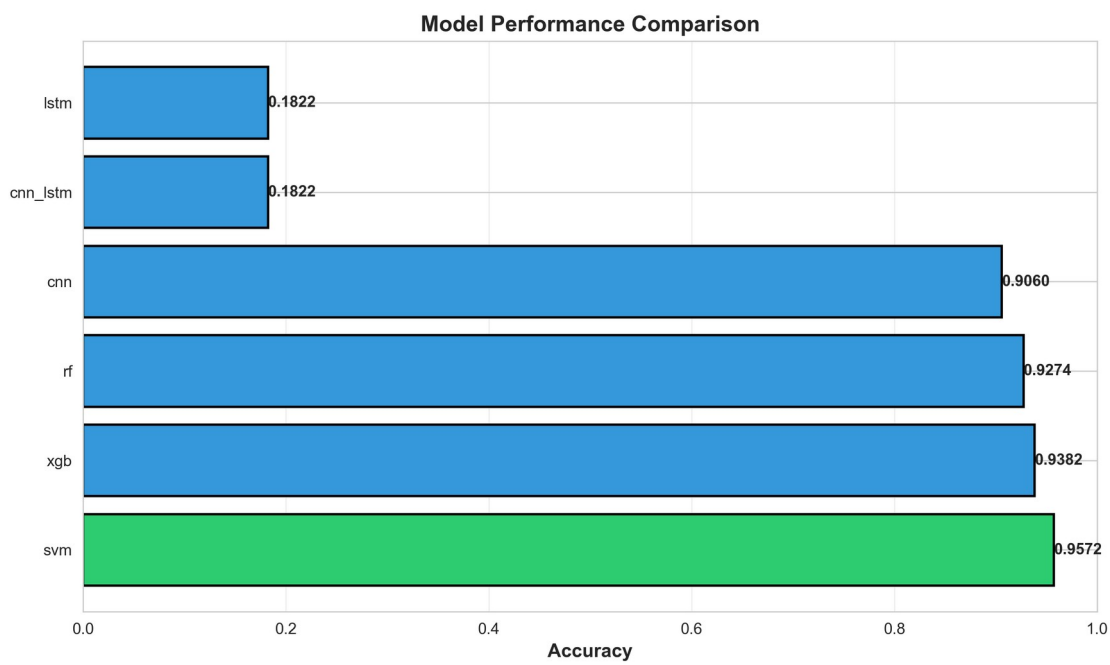


Figure 21

Figure 22. Accuracy vs. training epoch for the deep learning model(s).

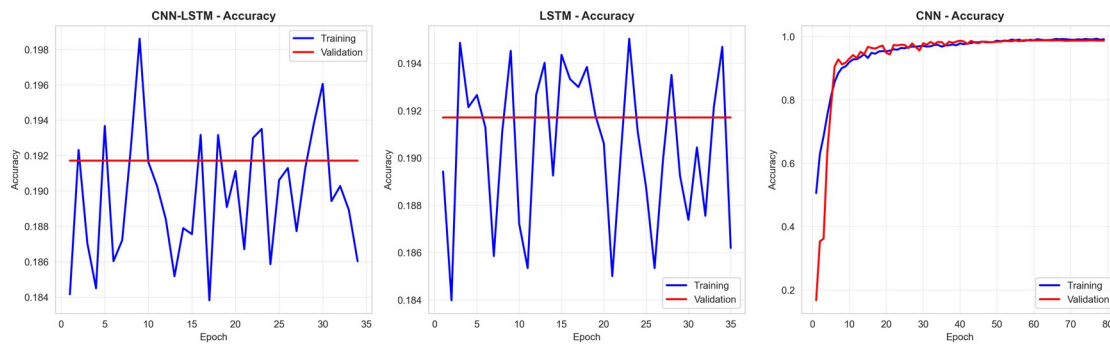


Figure 22

29. References

1. D. Anguita, A. Ghio, L. Oneto, X. Parra, and J. L. Reyes-Ortiz, "Human activity recognition on smartphones using a multiclass hardware-friendly support vector machine," *IEEE Sensors Journal*, vol. 13, no. 9, pp. 3416–3424, Sept. 2013.
2. F. J. Ordóñez and D. Roggen, "Deep convolutional and LSTM recurrent neural networks for multimodal wearable activity recognition," *Sensors*, vol. 16, no. 1, pp. 1–25, Jan. 2016.
3. O. D. Lara and M. A. Labrador, "A survey on human activity recognition using wearable sensors," *IEEE Communications Surveys & Tutorials*, vol. 15, no. 3, pp. 1192–1209, Third Quarter 2013.
4. C. A. Ronao and S. B. Cho, "Human activity recognition with smartphone sensors using deep learning neural networks," *Expert Systems with Applications*, vol. 59, pp. 235–244, Oct. 2016.
5. M. Zeng, L. T. Nguyen, B. Yu, O. J. Mengshoel, J. Zhu, P. Wu, and J. Zhang, "Convolutional neural networks for human activity recognition using mobile sensors," in *Proc. 6th Int. Conf. on Mobile Computing, Applications and Services (MobiCASE)*, Austin, TX, USA, 2014, pp. 197–205.
6. Y. Chen, K. Xue, R. Chen, Y. Zhang, and L. Guo, "Sensor-based human activity recognition via deep learning," *Neurocomputing*, vol. 337, pp. 121–132, Apr. 2019.

This report was generated entirely from the supplied project artifacts (plots, visualizations, evaluation reports, and metric files) plus the supporting PowerPoint presentation. No numerical results were invented; items requested by the report template but absent from the supplied files are explicitly flagged as missing throughout (Sections 7, 8, 13, 14, 15, 16, 22, 26).